

5 op T2 - raaka-datan merkityksellisten ominaisuuksien/piirteiden tarkastelu (ml. luokittelu/regressio) ja Power Appsin luominen

Antti Kiviniemi, 249780

Valitse jokin vapaavalintainen tietoaaineisto ja jokin vapaavalintainen autoML-paketti (katso linkit tehtäväksiannon alapuolella).

Muodosta Colab-tiedosto, jonka avulla teet joko luokittelijan tai regressiomallin. (3 pistettä)

Tarkastele piirteitä myös WHAT-IF-analyysillä. (2 pistettä)

Valitse jokin vapaavalintainen tietoaaineisto ja jokin vapaavalintainen autoML-paketti

UCI, Machine Learning Repository, Mushroom Data Set

URL: <https://archive.ics.uci.edu/ml/datasets/mushroom>

Linkki dataan: <https://archive.ics.uci.edu/ml/machine-learning-databases/mushroom/agaricus-lepiota.data>

Linkki datan kuvaukseen: <https://archive.ics.uci.edu/ml/machine-learning-databases/mushroom/agaricus-lepiota.names>

Power BI:stä pieni kuva datasta:

Queries [3]	= Table.TransformColumnTypes(edible_poisonous_Table,{{"edible_poisonous", type text}},				
mushroom	A ^B _C edible_poisonous	A ^B _C cap_shape	A ^B _C cap_surface	A ^B _C cap_color	A ^B _C bruises
mushroom_malli_kaikki	966 e	f	s	g	f
mushroom_valitut	967 p	f	s	n	t
	968 e	x	f	n	t
	969 e	x	s	n	f
	970 e	x	f	w	f
	971 e	f	f	n	f
	972 e	x	f	n	t
	973 e	f	f	n	t
	974 e	x	f	g	f
	975 e	f	f	n	t
	976 e	x	s	g	f
	977 e	b	y	w	t
	978 e	x	f	n	t
	979 e	x	y	y	t
	980 e	f	f	g	f
	981 e	f	f	n	t
	982 e	x	s	y	t

Muodosta Colab-tiedosto, jonka avulla teet joko luokittelijan tai regressiomallin.

Luokittelijan teko

```
# Lue lisää Pandas-paketin io-toiminnallisuuksista https://pandas.pydata.org/pandas-docs/stable/user\_guide/io.html
import pandas as pd

# Luetaan agaricus-leiota.data (sienien data)

csv_path = 'https://archive.ics.uci.edu/ml/machine-learning-databases/mushroom/agaricus-leiota.data'

# Nimetään sarakkeet

csv_columns = [
    "edible_poisonous", "cap_shape", "cap_surface", "cap_color", "bruises", "odor",
    "gill-attachment", "gill-spacing", "gill-size", "gill-color", "stalk-shape",
    "stalk-root", "stalk-surface-above-ring", "stalk-surface-below-ring",
    "stalk-color-above-ring", "stalk-color-below-ring", "veil-type", "veil-color",
    "ring-number", "ring-type", "spore-print-color", "population", "habitat"]

#luetaan data ja käytetään pilkkua erottimena
dataset = pd.read_csv(csv_path, names=csv_columns, delimiter=',')

# Muodotetaan csv-tiedosto
dataset.to_csv('UCI_mushroom_colab.csv')
```

```
[2] # Tulostetaan muutama rivi
    dataset.shape
```

(8124, 23)

```
# Jaetaan datajoukko kahteen osaan
data = dataset.sample(frac=0.95, random_state=786)
data_unseen = dataset.drop(data.index)
data.reset_index(inplace=True, drop=True)
data_unseen.reset_index(inplace=True, drop=True)
print('Mallinnukseen käytettävä data: ' + str(data.shape))
print('Ennakointiin käytettävä data: ' + str(data_unseen.shape))
```

➡ Mallinnukseen käytettävä data: (7718, 23)
Ennakointiin käytettävä data: (406, 23)

```
# Asennetaan PyCaret
!pip install pycaret
```

```
[5] # Ladataan paketit, joita PyCaret tarvitsee
import numpy as np
import warnings
```

```
# Ladataan PyCaret-paketista kaikki luokittelijavaihtoehdot
from pycaret.classification import *
```

```
# Muodostetaan luokittelijat
#clf=setup(data=data,target='Over-50K')
clf=setup(data=data,target='edible_poisonous')
```

Setup Successfully Completed!

	Description	Value
0	session_id	7062
1	Target Type	Binary
2	Label Encoded	e: 0, p: 1
3	Original Data	(7718, 23)
4	Missing Values	False
5	Numeric Features	0
6	Categorical Features	22
7	Ordinal Features	False

```
# Vertaa luokittelijoita
compare_models()
```

Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT (Sec)
0 Quadratic Discriminant Analysis	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.0785
1 Extra Trees Classifier	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.3515
2 Light Gradient Boosting Machine	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.1319
3 Decision Tree Classifier	0.9998	0.9998	0.9996	1.0000	0.9998	0.9996	0.9996	0.0223
4 Random Forest Classifier	0.9998	1.0000	0.9996	1.0000	0.9998	0.9996	0.9996	0.1130
5 Ada Boost Classifier	0.9998	1.0000	0.9996	1.0000	0.9998	0.9996	0.9996	0.4576
6 Gradient Boosting Classifier	0.9998	1.0000	0.9996	1.0000	0.9998	0.9996	0.9996	1.1308
7 CatBoost Classifier	0.9998	1.0000	0.9996	1.0000	0.9998	0.9996	0.9996	3.2659
8 Logistic Regression	0.9996	1.0000	0.9992	1.0000	0.9996	0.9993	0.9993	0.1185
9 K Neighbors Classifier	0.9996	1.0000	0.9992	1.0000	0.9996	0.9993	0.9993	0.0905
10 SVM - Linear Kernel	0.9996	0.0000	0.9992	1.0000	0.9996	0.9993	0.9993	0.0236
11 Ridge Classifier	0.9996	0.0000	0.9992	1.0000	0.9996	0.9993	0.9993	0.0260
12 Extreme Gradient Boosting	0.9996	1.0000	0.9992	1.0000	0.9996	0.9993	0.9993	1.0161
13 Linear Discriminant Analysis	0.9994	0.9994	0.9989	1.0000	0.9994	0.9989	0.9989	0.1726
14 Naive Bayes	0.9596	0.9962	0.9992	0.9237	0.9599	0.9194	0.9224	0.0144

[illegible]



```
# Lisäinformaatiota malleista löytyy, kun hakee Googlestä Reference-sarakkeessa mainittua models()
```



ID	Name	Reference	Turbo
lr	Logistic Regression	<code>sklearn.linear_model.LogisticRegression</code>	True
knn	K Neighbors Classifier	<code>sklearn.neighbors.KNeighborsClassifier</code>	True
nb	Naive Bayes	<code>sklearn.naive_bayes.GaussianNB</code>	True
dt	Decision Tree Classifier	<code>sklearn.tree.DecisionTreeClassifier</code>	True
svm	SVM - Linear Kernel	<code>sklearn.linear_model.SGDClassifier</code>	True
rbfsvm	SVM - Radial Kernel	<code>sklearn.svm.SVC</code>	False
gpc	Gaussian Process Classifier	<code>sklearn.gaussian_process.GPC</code>	False
mlp	MLP Classifier	<code>sklearn.neural_network.MLPClassifier</code>	False
ridge	Ridge Classifier	<code>sklearn.linear_model.RidgeClassifier</code>	True
rf	Random Forest Classifier	<code>sklearn.ensemble.RandomForestClassifier</code>	True
qda	Quadratic Discriminant Analysis	<code>sklearn.discriminant_analysis.QDA</code>	True
ada	Ada Boost Classifier	<code>sklearn.ensemble.AdaBoostClassifier</code>	True
gbc	Gradient Boosting Classifier	<code>sklearn.ensemble.GradientBoostingClassifier</code>	True
lda	Linear Discriminant Analysis	<code>sklearn.discriminant_analysis.LDA</code>	True
et	Extra Trees Classifier	<code>sklearn.ensemble.ExtraTreesClassifier</code>	True
xgboost	Extreme Gradient Boosting	xgboost.readthedocs.io	True
lightgbm	Light Gradient Boosting Machine	github.com/microsoft/LightGBM	True
catboost	CatBoost Classifier	catboost.ai	True

```
# Listataan mallien lyhenteet
model_metadata = models()
model_metadata['Name']
```

```

↳ ID                               Logistic Regression
lr                                K Neighbors Classifier
knn                               Naive Bayes
nb                                Decision Tree Classifier
dt                                SVM - Linear Kernel
svm                               SVM - Radial Kernel
rbfsvm                           Gaussian Process Classifier
gpc                               MLP Classifier
mlp                               Ridge Classifier
ridge                             Random Forest Classifier
rf                                Quadratic Discriminant Analysis
qda                               Ada Boost Classifier
ada                              Gradient Boosting Classifier
gbc                              Linear Discriminant Analysis
lda                              Extra Trees Classifier
et                               Extreme Gradient Boosting
xgboost                          Light Gradient Boosting Machine
lightgbm                         CatBoost Classifier
catboost
Name: Name, dtype: object

```

Luodaan luokitellijat, jotka ovat merkitty keltaisella `compare_models()` tuloksissa. Valitsin kuitenkin mielenkiinnosta pari mallia, joiden tarkkuus ei ole 100.

[illegible][illegible]

```
[12] catboost = create_model('catboost')
```

	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
1	0.9982	0.9999	0.9962	1.0000	0.9981	0.9963	0.9963
2	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
4	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
5	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
6	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
7	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
8	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
9	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Mean	0.9998	1.0000	0.9996	1.0000	0.9998	0.9996	0.9996
SD	0.0006	0.0000	0.0011	0.0000	0.0006	0.0011	0.0011

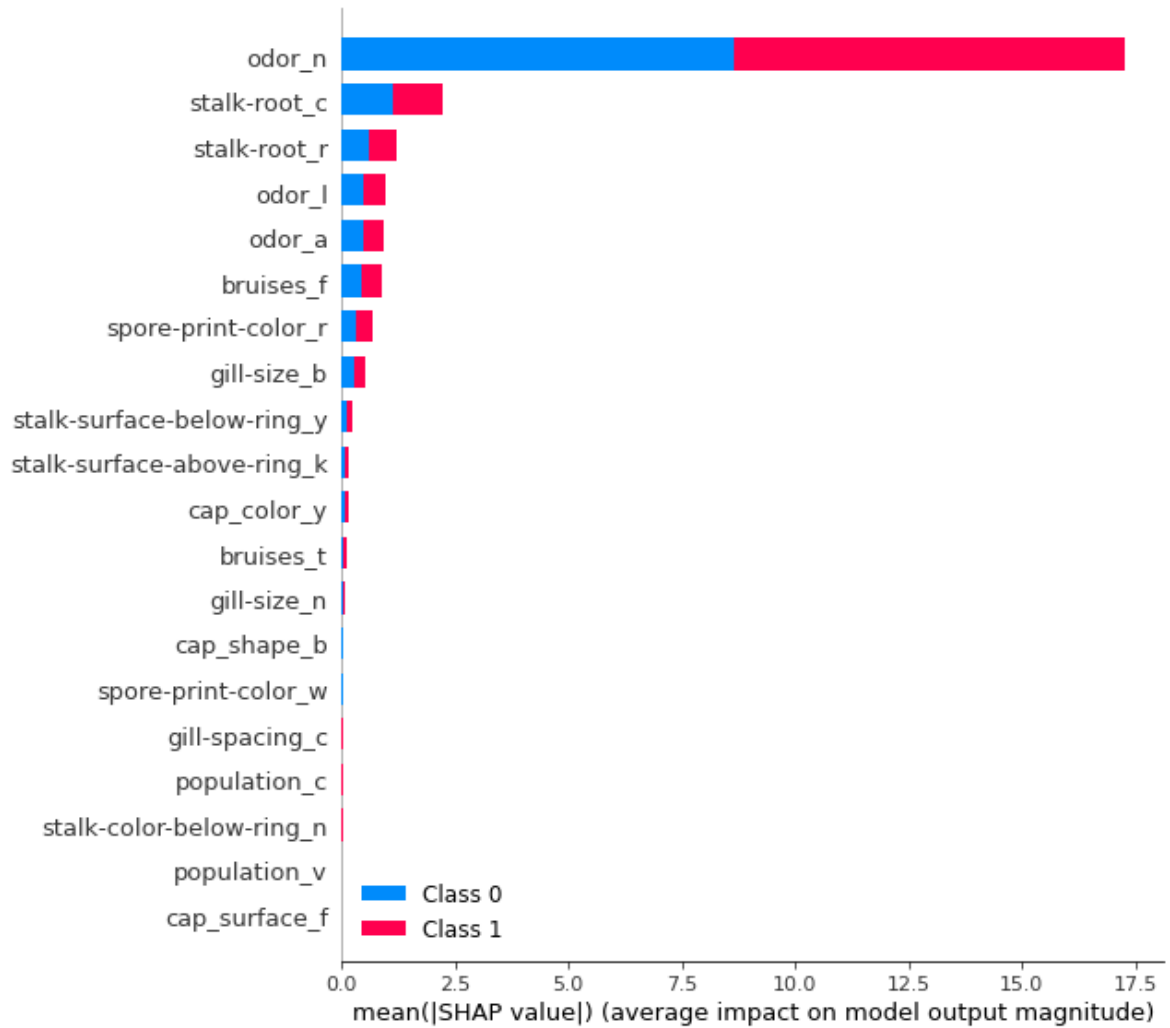
```
dt = create_model('dt')
```

	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
1	0.9982	0.9981	0.9962	1.0000	0.9981	0.9963	0.9963
2	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
4	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
5	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
6	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
7	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
8	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
9	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Mean	0.9998	0.9998	0.9996	1.0000	0.9998	0.9996	0.9996
SD	0.0006	0.0006	0.0011	0.0000	0.0006	0.0011	0.0011

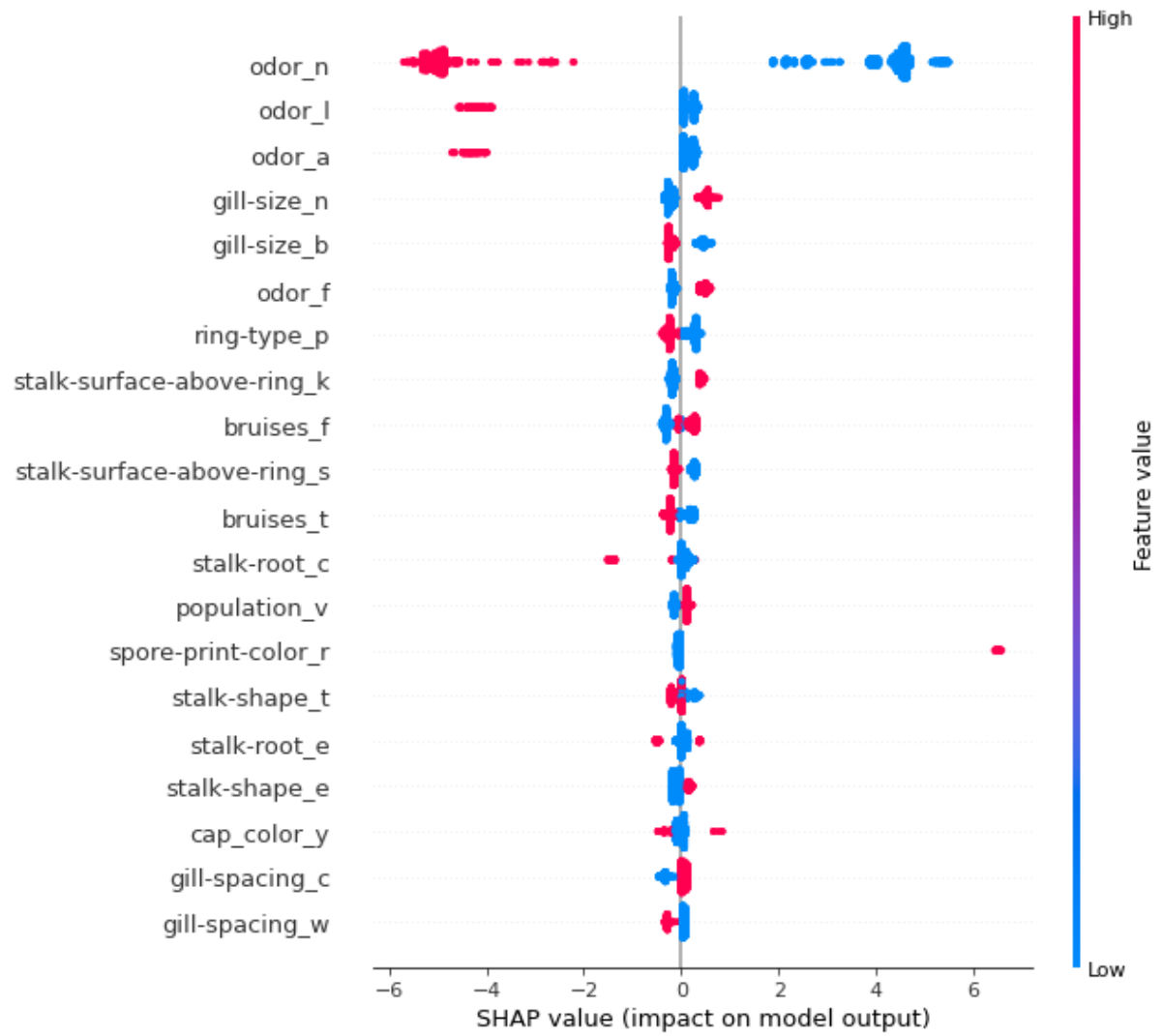
Tulkitaan luokittelijoihin vaikuttavia tekijöitä ennen kuin "tuunataan"/viritetään niitä

```
# Lue lisää https://shap.readthedocs.io/en/latest/  
!pip install shap
```

interpret_model(lightgbm)

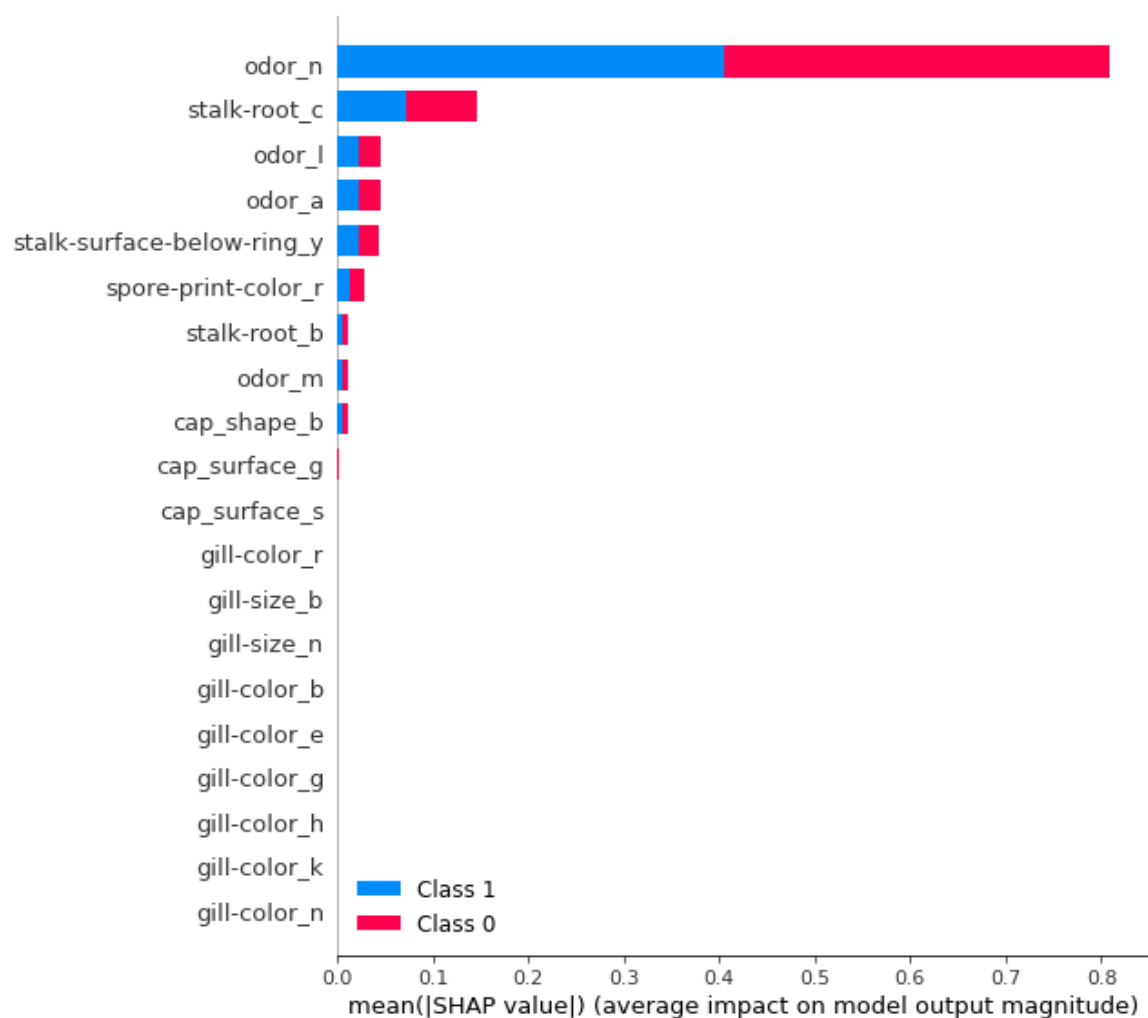


 interpret_model(catboost)





interpret_model(dt)



"Tuunataan"/viritetään luodut mallit hyperparametrien suhteen

```
▶ tuned_lightgbm = tune_model(lightgbm)
```

[illegible]

```
[19] tuned_catboost = tune_model(catboost)
```

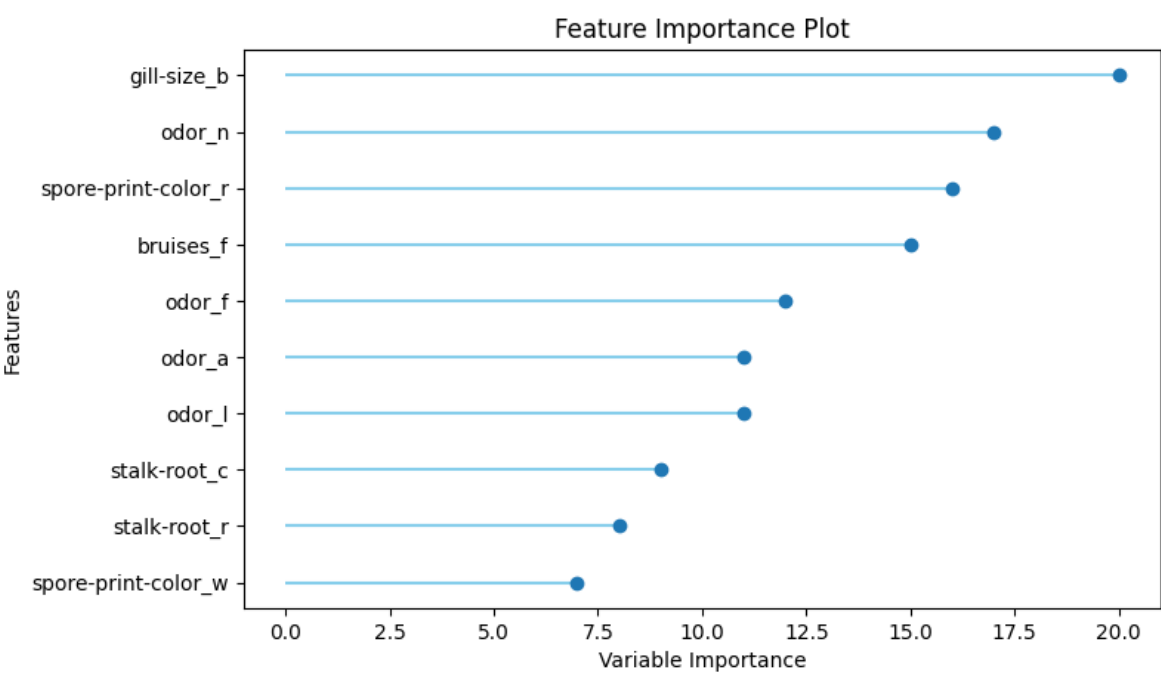
	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
1	0.9982	1.0000	0.9962	1.0000	0.9981	0.9963	0.9963
2	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
4	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
5	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
6	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
7	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
8	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
9	0.9981	1.0000	0.9962	1.0000	0.9981	0.9963	0.9963
Mean	0.9996	1.0000	0.9992	1.0000	0.9996	0.9993	0.9993
SD	0.0007	0.0000	0.0015	0.0000	0.0008	0.0015	0.0015

```
tuned_dt = tune_model(dt)
```

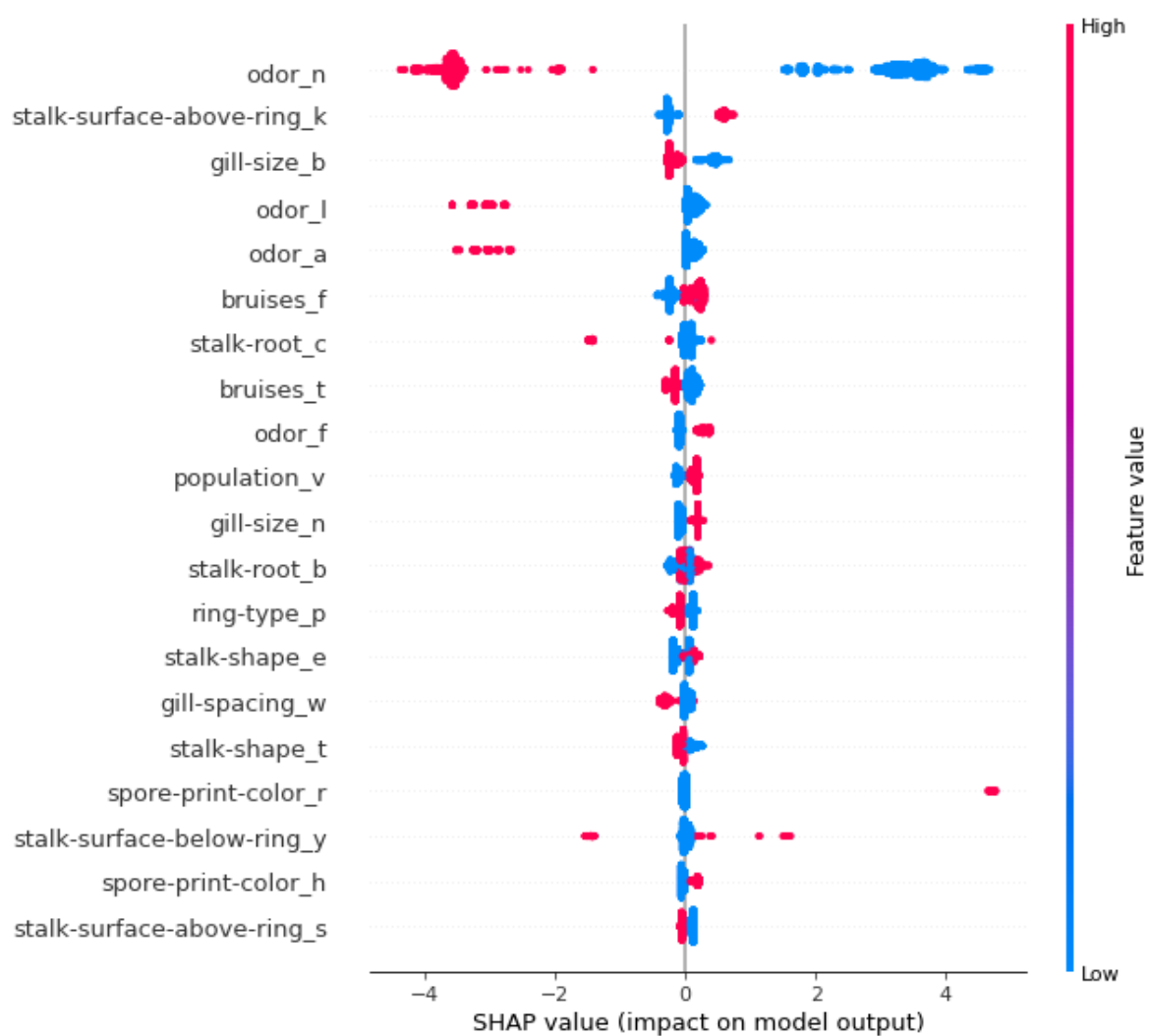
	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
1	0.9982	0.9981	0.9962	1.0000	0.9981	0.9963	0.9963
2	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
4	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
5	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
6	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
7	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
8	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
9	0.9981	1.0000	0.9962	1.0000	0.9981	0.9963	0.9963
Mean	0.9996	0.9998	0.9992	1.0000	0.9996	0.9993	0.9993
SD	0.0007	0.0006	0.0015	0.0000	0.0008	0.0015	0.0015

Tarkastellaan piirteitä, joita luokittelijat käyttävät

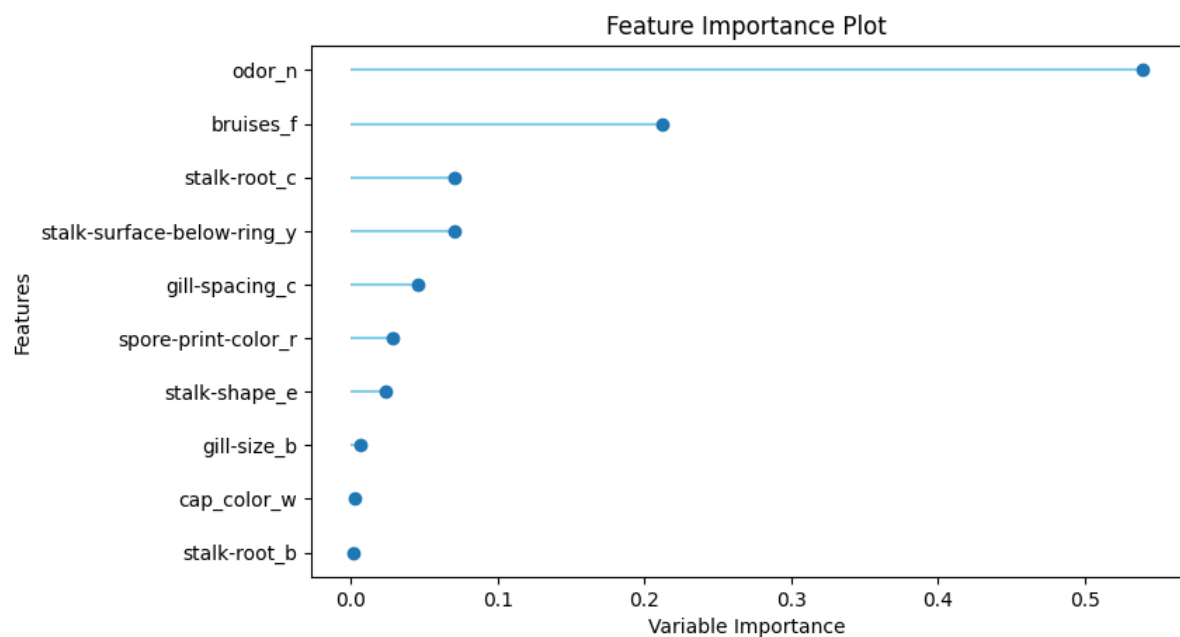
```
plot_model(tuned_lightgbm, plot='feature')
```



interpret_model(tuned_catboost)



```
plot_model(tuned_dt, plot='feature')
```



Kokeillaan mallien toimivuutta

```
[26] predict_model(tuned_lightgbm)
```

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	Light Gradient Boosting Machine	0.9978	1.0	0.9955	1.0	0.9978	0.9957	0.9957



```
predict_model(tuned_catboost)
```



	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	CatBoost Classifier	0.9987	1.0	0.9973	1.0	0.9987	0.9974	0.9974



```
predict_model(tuned_dt);
```



	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	Decision Tree Classifier	1.0	1.0	1.0	1.0	1.0	1.0	1.0

Viimeistellään valittu luokittelija koskemaan koko datajoukkoa ja kokeillaan luokittelua siihen data_unseen-nimiseen datajoukkoon, jota ei käytetty luokittelijan muodostamisessa

```
[31] final_lightgbm = finalize_model(tuned_lightgbm)
```

```
[32] print(final_lightgbm)
```

```
LGBMClassifier(boosting_type='gbdt', class_weight=None, colsample_bytree=1.0,
importance_type='split', learning_rate=0.4, max_depth=70,
min_child_samples=20, min_child_weight=0.001, min_split_gain=0,
n_estimators=30, n_jobs=-1, num_leaves=20, objective=None,
random_state=7062, reg_alpha=0.2, reg_lambda=0.7, silent=True,
subsample=1.0, subsample_for_bin=200000, subsample_freq=0)
```

```
unseen_predictions = predict_model(final_lightgbm, data=data_unseen)
unseen_predictions.head()
```

l- t	gill- spacing	gill- size	gill- color	stalk- shape	stalk- root	stalk- surface- above- ring	stalk- surface- below- ring	stalk- color- above- ring	stalk- color- below- ring	veil- type	veil- color	ring- number	ring- type	spore- print- color	population	habitat	Label	Score
f	c	b	n	e	c	s	s	w	w	p	w	o	p	k	n	g	e	0.0003
f	c	b	n	e	c	s	s	w	w	p	w	o	p	k	s	m	e	0.0003
f	c	n	k	e	e	s	s	w	w	p	w	o	p	n	v	u	p	0.9994
f	c	n	n	e	e	s	s	w	w	p	w	o	p	n	s	u	p	0.9994
f	c	b	w	e	c	s	s	w	w	p	w	o	p	n	n	g	e	0.0003

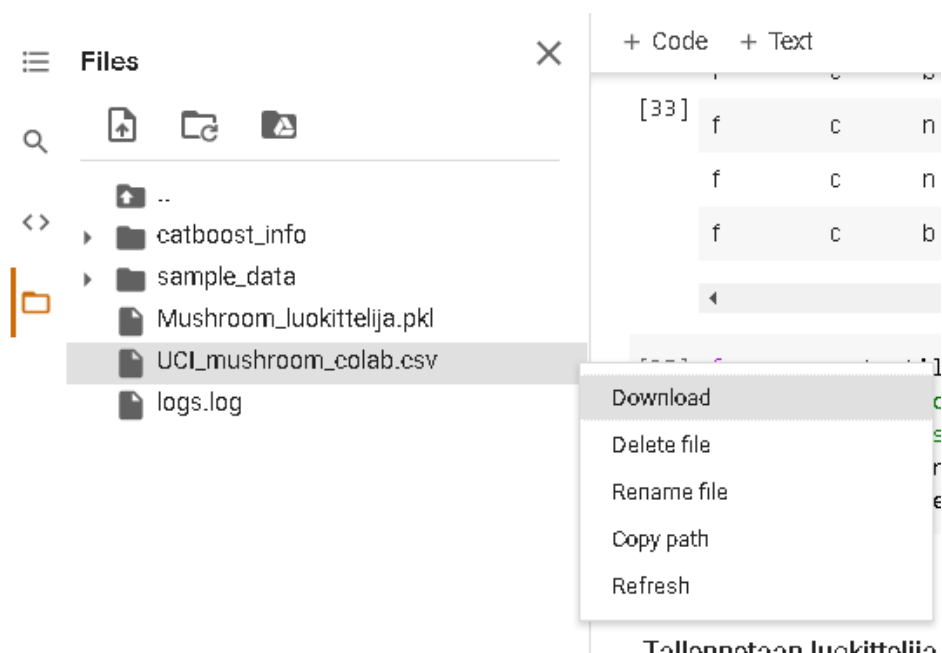
```
from pycaret.utils import check_metric
unseen_predictions.rename(columns = {'edible_poisonous':'Origin'}, inplace = True)
check_metric(unseen_predictions.Origin, unseen_predictions.Label, 'Accuracy')
```

1.0

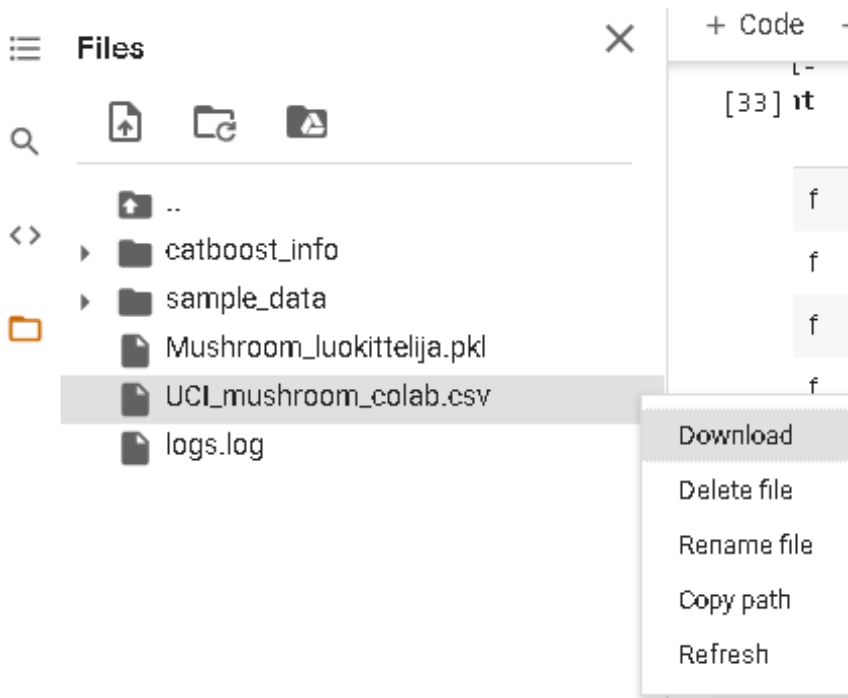
```
save_model(final_lightgbm, 'Mushroom_luokittelija')
```

Transformation Pipeline and Model Succesfully Saved

Ladataan luokittelija koneelle myöhempää käyttöä varten



Ladataan colabissa luotu datajoukko myöhempää Power BI käsittelyä varten

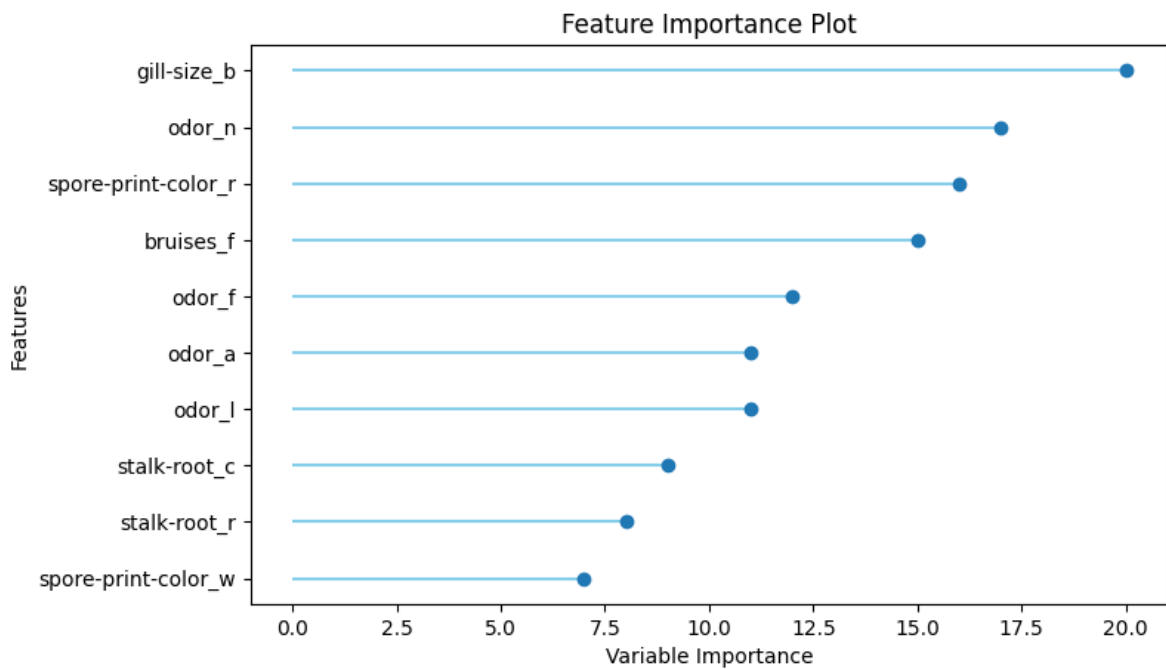


Tarkastele piirteitä myös WHAT-IF-analyysillä. (2 pistettä)

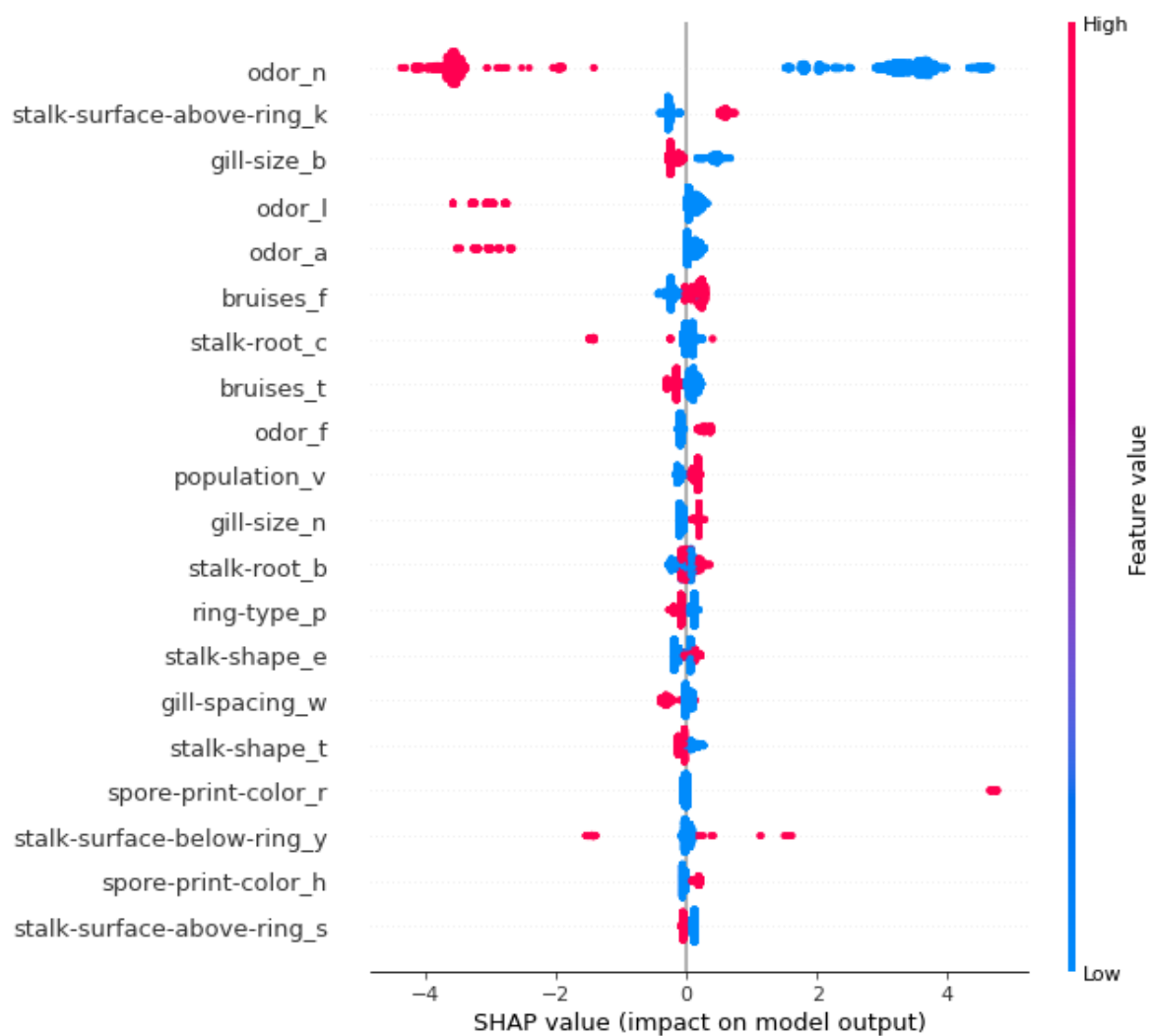
Valitaan datasta tärkeimmät piirteet, joiden perusteella muodostetaan WHAT-IF-analyysi (ja myöhemmin PowerApp)

Tarkastellaan piirteitä, joita luokittelijat käyttävät

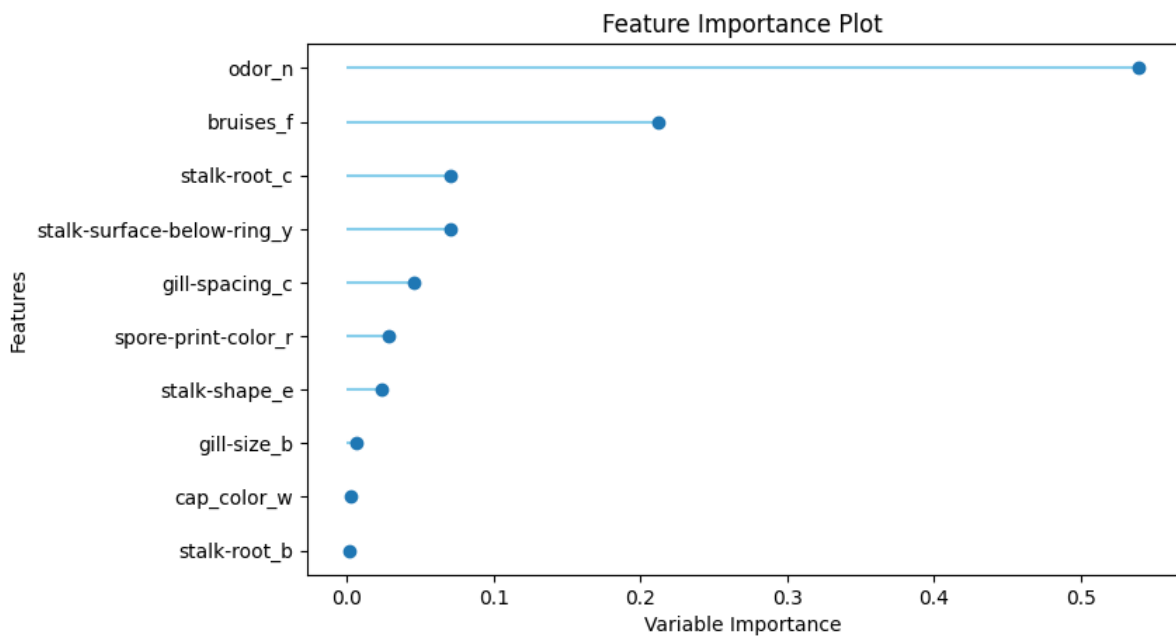
```
plot_model(tuned_lightgbm, plot='feature')
```



interpret_model(tuned_catboost)



```
plot_model(tuned_dt, plot='feature')
```



Tärkeimmät piirteet luokittelijoiden mukaan:

- lightgbm
 - gill-size
 - odor
 - spore-print-color
 - bruises
 - stalk-root
 - spore-print-color
- catboost
 - odor
 - stalk-surface-above-ring
 - stalk-root
 - stalk-surface-below-ring
 - spore-print-color
- dt (decisiontree)
 - odor
 - bruises
 - stalk-root
 - stalk-surface-below-ring
 - gill-spacing
 - spore-print-color
 - stalk-shape
- Valitut piirteet What-if -analyysiin (9 piirrettä)
 - bruises

- gill-size
- gill-spacing
- odor
- spore-print-color
- stalk-root
- stalk-shape
- stalk-surface-above-ring
- stalk-surface-below-ring
- Valitut piirteet Power Appiin (4 piirrettä) -> 4 piirrettä yhdestä luokittelijasta -> lightgbm
 - gill-size
 - odor
 - spore-print-color
 - bruises

Power BI:ssä luodaan uusi datajoukko, jossa on valitut piirteet datajoukosta, joita käytetään What If -analyysiin

	A_C^B edible_poisonous	A_C^B bruises	A_C^B odor	A_C^B gill-spacing	A_C^B gill-size
1	p	t	p	c	n
2	e	t	a	c	b
3	e	t	l	c	b
4	p	t	p	c	n
5	e	f	n	w	b
6	e	t	a	c	b
7	e	t	a	c	b
8	e	t	l	c	b
9	p	t	p	c	n
10	e	t	a	c	b

What If -analyysi colab tiedostolla

```
[1] #title Install the What-If Tool widget if running in Colab {display-mode: "form"}

# If running in Colab then pip install, otherwise no need.
try:
    import google.colab
    !pip install --upgrade witwidget
except Exception:
    pass
```

```

#title Define helper functions {display-mode: "form"}

import pandas as pd
import numpy as np
import tensorflow as tf
import functools

# Creates a tf feature spec from the dataframe and columns specified.
def create_feature_spec(df, columns=None):
    feature_spec = {}
    if columns == None:
        columns = df.columns.values.tolist()
    for f in columns:
        if df[f].dtype is np.dtype(np.int64):
            feature_spec[f] = tf.io.FixedLenFeature(shape=(), dtype=tf.int64)
        elif df[f].dtype is np.dtype(np.float64):
            feature_spec[f] = tf.io.FixedLenFeature(shape=(), dtype=tf.float32)
        else:
            feature_spec[f] = tf.io.FixedLenFeature(shape=(), dtype=tf.string)
    return feature_spec

# Creates simple numeric and categorical feature columns from a feature spec and a
# list of columns from that spec to use.
#
# NOTE: Models might perform better with some feature engineering such as bucketed
# numeric columns and hash-bucket/embedding columns for categorical features.
def create_feature_columns(columns, feature_spec):
    ret = []
    for col in columns:
        if feature_spec[col].dtype is tf.int64 or feature_spec[col].dtype is tf.float32:
            ret.append(tf.feature_column.numeric_column(col))
        else:
            ret.append(tf.feature_column.indicator_column(
                tf.feature_column.categorical_column_with_vocabulary_list(col, list(df[col].unique()))))
    return ret

```

```

# An input function for providing input to a model from tf.Examples
def tfexamples_input_fn(examples, feature_spec, label, mode=tf.estimator.ModeKeys.EVAL,
                        num_epochs=None,
                        batch_size=64):
    def ex_generator():
        for i in range(len(examples)):
            yield examples[i].SerializeToString()
    dataset = tf.data.Dataset.from_generator(
        ex_generator, tf.dtypes.string, tf.TensorShape([]))
    if mode == tf.estimator.ModeKeys.TRAIN:
        dataset = dataset.shuffle(buffer_size=2 * batch_size + 1)
    dataset = dataset.batch(batch_size)
    dataset = dataset.map(lambda tf_example: parse_tf_example(tf_example, label, feature_spec))
    dataset = dataset.repeat(num_epochs)
    return dataset

# Parses Tf.Example protos into features for the input function.
def parse_tf_example(example_proto, label, feature_spec):
    parsed_features = tf.io.parse_example(serialized=example_proto, features=feature_spec)
    target = parsed_features.pop(label)
    return parsed_features, target

# Converts a dataframe into a list of tf.Example protos.
def df_to_examples(df, columns=None):
    examples = []
    if columns == None:
        columns = df.columns.values.tolist()
    for index, row in df.iterrows():
        example = tf.train.Example()
        for col in columns:
            if df[col].dtype is np.dtype(np.int64):
                example.features.feature[col].int64_list.value.append(int(row[col]))
            elif df[col].dtype is np.dtype(np.float64):
                example.features.feature[col].float_list.value.append(row[col])
            elif row[col] == row[col]:
                example.features.feature[col].bytes_list.value.append(row[col].encode('utf-8'))
        examples.append(example)
    return examples

```

```

# Converts a dataframe column into a column of 0's and 1's based on the provided test.
# Used to force label columns to be numeric for binary classification using a TF estimator.
def make_label_column_numeric(df, label_column, test):
    df[label_column] = np.where(test(df[label_column]), 1, 0)

```

```

#Read training dataset from CSV {display-mode: "form"}

import pandas as pd

# Set the path to the CSV containing the dataset to train on.
csv_path = 'https://archive.ics.uci.edu/ml/machine-learning-databases/mushroom/agaricus-lepiota.data'

# Set the column names for the columns in the CSV. If the CSV's first line is a header line containing
# the column names, then set this to None.
csv_columns = [
    "edible_poisonous", "cap_shape", "cap_surface", "cap_color", "bruises", "odor",
    "gill-attachment", "gill-spacing", "gill-size", "gill-color", "stalk-shape",
    "stalk-root", "stalk-surface-above-ring", "stalk-surface-below-ring",
    "stalk-color-above-ring", "stalk-color-below-ring", "veil-type", "veil-color",
    "ring-number", "ring-type", "spore-print-color", "population", "habitat"]

# Read the whole dataset so we can create 2 different datasets
dataframe = pd.read_csv(csv_path, names=csv_columns, delimiter=',')

```

```

# print the dataset
dataframe

```



	edible_poisonous	cap_shape	cap_surface	cap_color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	stalk-shape	stalk-root	stalk-surface-above-ring	stalk-surface-below-ring
0	p	x	s	n	t	p	f	c	n	k	e	e	s	s
1	e	x	s	y	t	a	f	c	b	k	e	c	s	s
2	e	b	s	w	t	l	f	c	b	n	e	c	s	s
3	p	x	y	w	t	p	f	c	n	n	e	e	s	s
4	e	x	s	g	f	n	f	w	b	k	t	e	s	s
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
8119	e	k	s	n	f	n	a	c	b	y	e	?	s	s
8120	e	x	s	n	f	n	a	c	b	y	e	?	s	s
8121	e	f	s	n	f	n	a	c	b	n	e	?	s	s
8122	p	k	y	n	f	y	f	c	n	b	t	?	s	k
8123	e	x	s	n	f	n	a	c	b	y	e	?	s	s

8124 rows x 23 columns


```
# Divide data into two parts: training and testing
df = dataframe.sample(frac=0.80, random_state=786)
df_unseen = dataframe.drop(df.index)
df.reset_index(inplace=True, drop=True)
df_unseen.reset_index(inplace=True, drop=True)
print('Mallinnukseen käytettävä data: ' + str(df.shape))
print('Ennakointiin käytettävä data: ' + str(df_unseen.shape))

# tulostetaan mallinnukseen käytettävä data
df
```

Mallinnukseen käytettävä data: (6499, 23)
Ennakointiin käytettävä data: (1625, 23)

	edible_poisonous	cap_shape	cap_surface	cap_color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	stalk-shape	stalk-root	stalk-surface-above-ring	stalk-surface-below-ring
0	p	f	y	y	f	f	f	c	b	p	e	b	k	k
1	e	x	s	n	f	n	f	w	b	p	t	e	s	s
2	e	x	f	n	t	n	f	c	b	w	t	b	s	s
3	p	f	s	w	t	p	f	c	n	n	e	e	s	s
4	e	x	f	g	f	n	f	w	b	p	e	?	k	s
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
6494	e	b	s	w	t	a	f	c	b	g	e	c	s	s
6495	p	f	s	b	t	f	f	c	b	h	t	b	s	f
6496	e	x	y	g	t	n	f	c	b	w	t	b	s	s
6497	p	x	s	w	t	f	f	c	b	w	t	b	f	f
6498	e	x	y	g	t	n	f	c	b	n	t	b	s	s

```
#Specify input columns and column to predict {display-mode: "form"}
import numpy as np

# Set the column in the dataset you wish for the model to predict
label_column = 'edible_poisonous'

# Make the label column numeric (0 and 1), for use in our model.
# In this case, examples with a target value of '>50K' are considered to be in
# the '1' (positive) class and all other examples are considered to be in the
# '0' (negative) class.
make_label_column_numeric(df, label_column, lambda val: val == 'p')

input_features = [
    'cap_shape', 'cap_surface', 'cap_color', 'bruises', 'odor',
    'gill-attachment', 'gill-spacing', 'gill-size', 'gill-color', 'stalk-shape',
    'stalk-root', 'stalk-surface-above-ring', 'stalk-surface-below-ring',
    'stalk-color-above-ring', 'stalk-color-below-ring', 'veil-type', 'veil-color',
    'ring-number', 'ring-type', 'spore-print-color', 'population', 'habitat']

# Create a list containing all input features and the label column
features_and_labels = input_features + [label_column]
```

```
[21] #Convert dataset to tf.Example protos {display-mode: "form"}
```

```
examples = df_to_examples(df)
```

```
[22] #Create and train the classifier {display-mode: "form"}
```

num_steps: 5000

```
num_steps = 5000 #@param {type: "number"}
```

```
# Create a feature spec for the classifier
```

```
feature_spec = create_feature_spec(df, features_and_labels)
```

```
# Define and train the classifier
```

```
train_inpf = functools.partial(tfexamples_input_fn, examples,
```

```
classifier = tf.estimator.LinearClassifier(
```

```
    feature_columns=create_feature_columns(input_features, fe  
classifier.train(train_inpf, steps=num_steps)
```

```
INFO:tensorflow:loss = 0.0002333207, step = 2200 (1.212 sec)  
INFO:tensorflow:global_step/sec: 80.397  
INFO:tensorflow:loss = 0.00017843636, step = 2300 (1.247 sec)  
INFO:tensorflow:global_step/sec: 82.5472  
INFO:tensorflow:loss = 0.00011800036, step = 2400 (1.209 sec)  
INFO:tensorflow:global_step/sec: 79.3679  
INFO:tensorflow:loss = 0.00012587386, step = 2500 (1.260 sec)  
INFO:tensorflow:global_step/sec: 80.3294  
INFO:tensorflow:loss = 0.00017583263, step = 2600 (1.245 sec)  
INFO:tensorflow:global_step/sec: 80.5046  
INFO:tensorflow:loss = 0.00013808801, step = 2700 (1.245 sec)  
INFO:tensorflow:global_step/sec: 82.7147  
INFO:tensorflow:loss = 0.000117313306, step = 2800 (1.208 sec)  
INFO:tensorflow:global_step/sec: 81.8305  
INFO:tensorflow:loss = 0.00010800487, step = 2900 (1.222 sec)  
INFO:tensorflow:global_step/sec: 79.759  
INFO:tensorflow:loss = 0.0001608686, step = 3000 (1.254 sec)  
INFO:tensorflow:global_step/sec: 82.5502  
INFO:tensorflow:loss = 0.00013264478, step = 3100 (1.209 sec)
```



```
#title Invoke What-If Tool for test data and the trained model {display-mode: "form"}
```

```
num_datapoints = 1620 #@param {type: "number"}
```

```
tool_height_in_px = 1000 #@param {type: "number"}
```

```
from witwidget.notebook.visualization import WitConfigBuilder
```

```
from witwidget.notebook.visualization import WitWidget
```

```
# Load up the test dataset
```

```
test_df = df_unseen
```

```
make_label_column_numeric(test_df, label_column, lambda val: val == 'p')
```

```
test_examples = df_to_examples(test_df[0:num_datapoints])
```

```
# Setup the tool with the test examples and the trained classifier
```

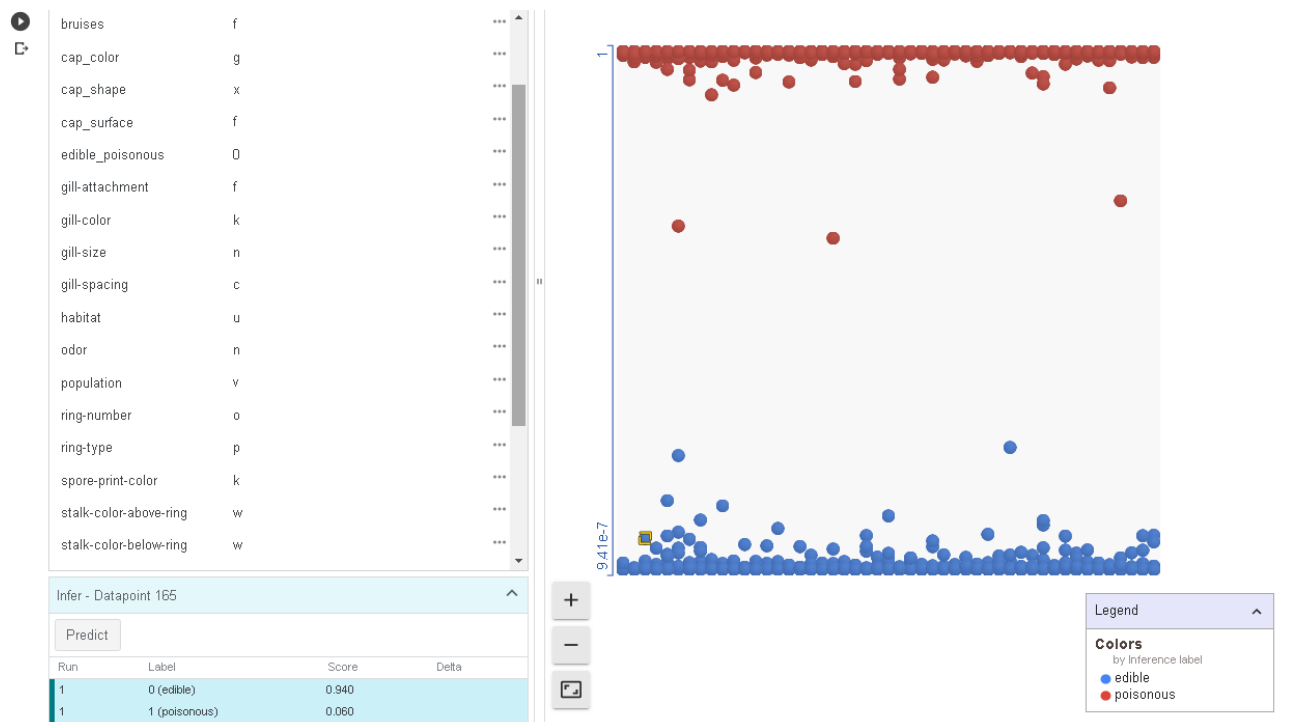
```
config_builder = WitConfigBuilder(test_examples).set_estimator_and_feature_spec(  
    classifier, feature_spec).set_label_vocab(['edible', 'poisonous'])
```

```
WitWidget(config_builder, height=tool_height_in_px)
```

num_datapoints: 1620

tool_height_in_px: 1000

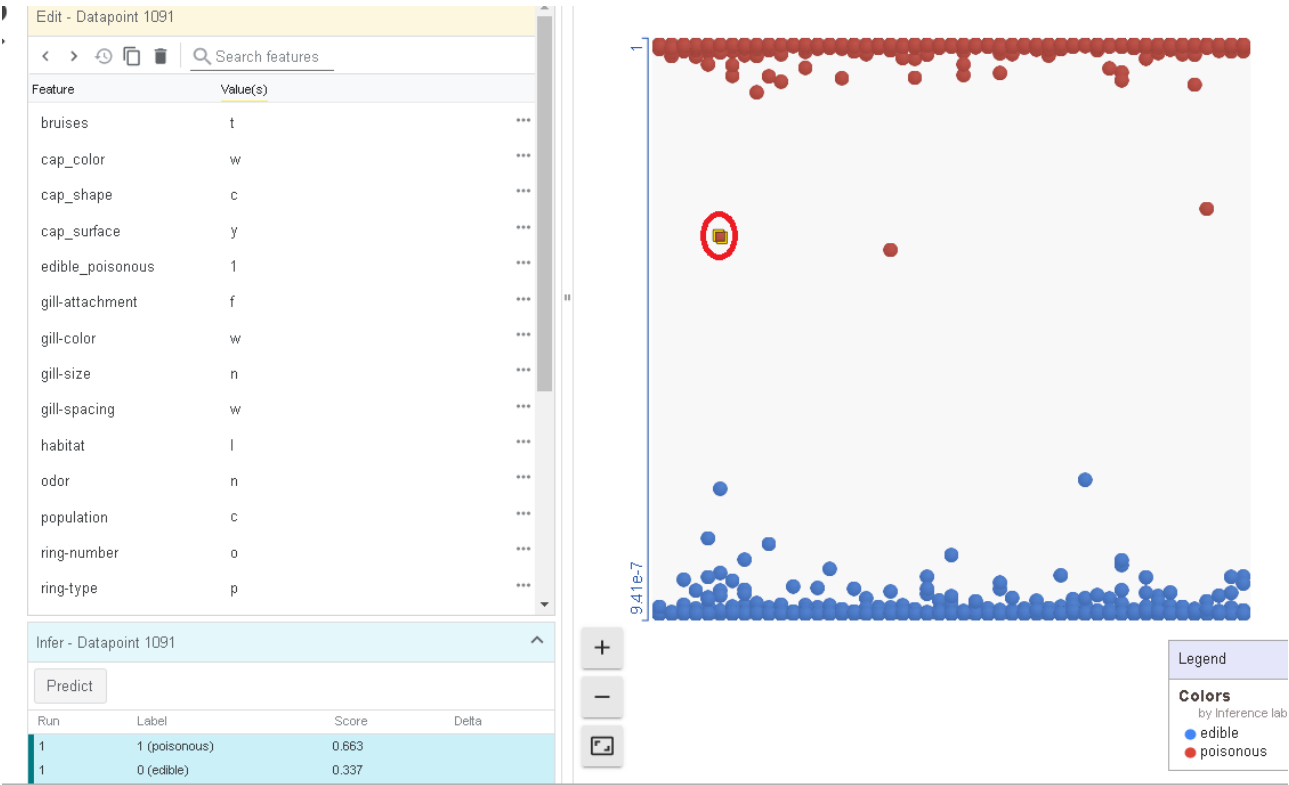
Tuloksena saatu What-If -työkalu



Ajan puutteen seurauksena jouduin ottamaan What-If-työkalussa tarkasteluun datasetin kaikki 23 arvoa. Olisin halunnut tarkastella vain 8:aa arvoa.

Datapisteen arvojen vaihtaminen:

Alkuperäiset arvot



Uudet arvot:

Alkuperäiset arvot

bruises	t
cap_color	w
cap_shape	c
cap_surface	y
edible_poisonous	1
gill-attachment	f
gill-color	w
gill-size	n
gill-spacing	w
habitat	l
odor	n
population	c
ring-number	o
ring-type	p

Uudet arvot

bruises	t
cap_color	w
cap_shape	f
cap_surface	s
edible_poisonous	0
gill-attachment	f
gill-color	w
gill-size	n
gill-spacing	w
habitat	d
odor	l
population	v
ring-number	o
ring-type	p

Ennuste / Ennusteen muutos:

Run	Label	Score	Delta
2	1 (poisonous)	0.708	↑ 0.045052
2	0 (edible)	0.292	↓ -0.045052
1	1 (poisonous)	0.663	
1	0 (edible)	0.337	

Arvojen vaihtaminen toisen kerran: spore-print-color arvon muuttaminen arvosta 'w' (= white) arvoon 'n' (= brown),

Ennusteen muutos: ennuste muuttuu myrkyllisestä siihen, että sieni on syötävä.

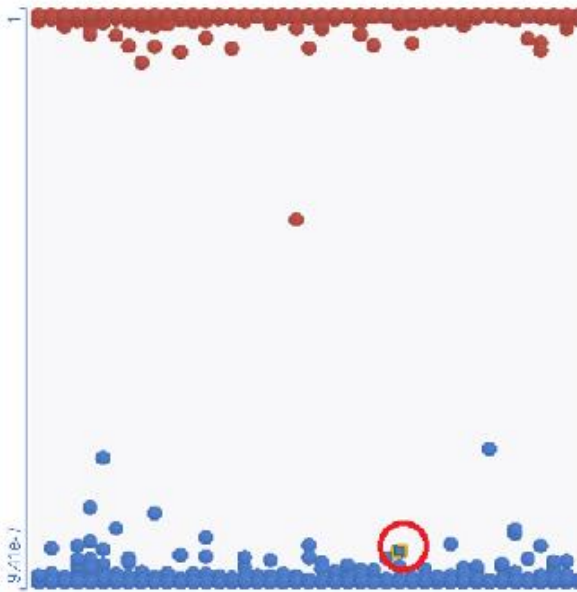
0 (edible)	0.991	↑ 0.699028
1 (poisonous)	0.009	↓ -0.699028

Kolmas arvojen muutos: datapisteen arvojen muutos odor: n (=none) tilaan f (= foul).

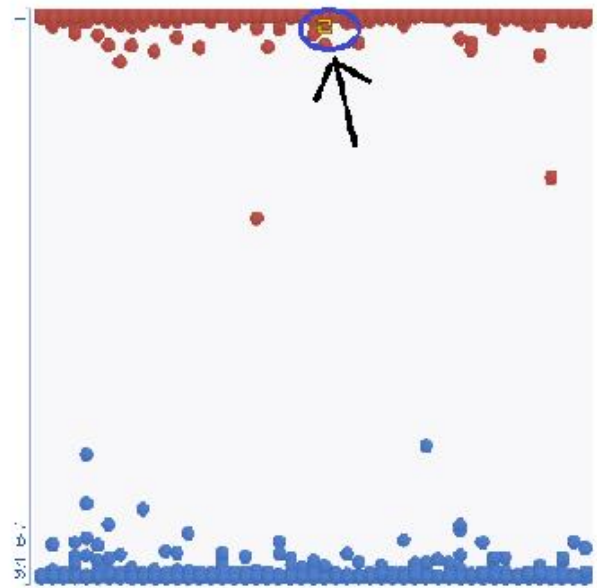
Ennusteen muutos: Muuttui syötävästä (0.055) myrkylliseksi (0.980).

2	1 (poisonous)	0.980	↑ 0.925875
2	0 (edible)	0.020	↓ -0.925875
1	0 (edible)	0.945	
1	1 (poisonous)	0.055	

Alkuperäinen sijainti



Uusi sijainti



Näiden perusteella olen huomannut, että esimerkiksi jos odor on f (= foul) ja spore-print on 'w' (= white) niin silloin on paljon todennäköisempää, että datapiste kuuluu myrkyllisten sienten joukkoon.